Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 Midterm Project — Research Report

Team: Victor Madu (individual project) **Date:** October 11, 2026

1. Problem Definition and Research Questions

LLMs now write a large share of online text, which raises a practical authorship question: does a response carry a recognizable “fingerprint” of the model family that produced it? We frame this as **supervised multiclass text classification**: given a user prompt (*LLM_Input*), a model response (*LLM_output*), or both, predict the generating family (*LLM_name*). Success on held-out data shows that stylistic and structural regularities exist in *this* dataset; it does not by itself make the classifier a reliable attribution tool for other models, prompts or time periods.

RQ1 — Can a CNN and an RNN (BiLSTM) identify the source family from the response alone? **RQ2** — Does the prompt help: input-only vs. output-only vs. input+output, for both architectures? **RQ3** (extra credit) — Do fingerprints transfer to task types unseen in training? **RQ4** (extra credit) — Which linguistic and structural properties separate the families, and does removing formatting hurt classification?

2. Dataset and Data Curation

Source. LMSYS *Chatbot Arena Conversations* (Zheng et al., 2023): ~33k pairwise comparisons, each with an original *question_id*, two anonymous model names and both dialogues (huggingface.co/datasets/lmsys/chatbot_arena_conversations; gated, access terms accepted). Prompts are CC BY 4.0 and model outputs CC BY-NC 4.0; raw data are **not** redistributed in our repository. The release predates current models (e.g. Gemini, Qwen, DeepSeek), so our label set is the 2023 landscape.

Construction of (LLM_name, LLM_Input, LLM_output). Each comparison yields two rows, one per side. Steps: (1) keep English conversations; (2) drop conversations flagged by the dataset’s OpenAI-moderation field; (3) concatenate user turns into *LLM_Input* and assistant turns into *LLM_output*; (4) map model names to a **family** (GPT; Claude; Llama-derived = Vicuna, Koala, Alpaca, LLaMA; ChatGLM; Google = PaLM/Bard) and discard unmapped or ambiguous names (e.g. GPT4All, which is not an OpenAI model); (5) remove empty/very short texts and **exact duplicate (family, response) pairs**, so canned replies cannot appear in both train and test; (6) drop families with fewer than 100 examples and cap each family at 1,600 rows with a fixed seed to keep classes reasonably balanced. “Llama-derived” mixes several fine-tunes, so a family is not a checkpoint.

Family	After cleaning	After cap	Train	Val	Test
ChatGLM	2968	1600	1120	160	320
Claude	5605	1600	1120	160	320
GPT	7667	1600	1120	160	320
Google	2465	1600	1120	160	320
Llama-derived	18763	1600	1120	160	320
Total	37468	8000	5600	800	1600

Table 1. Examples per family and split (from results/dataset_counts*.csv, split_counts.csv).

3. Experimental Design and Leakage Control

Grouped, stratified 70/10/20 split. Both answers to one *question_id* always land in the same partition (StratifiedGroupKFold), and the code asserts that no group crosses splits. Without this, a model could be rewarded for recognising a prompt it saw in training. The vocabulary (≤18,000 tokens) is fit on training text only; validation macro-F1 selects the epoch (best epoch restored); the test set is used once per model for reporting.

Conditions. *Input only*, *output only* and *input+output* (prompt, a <SEP> token, response), each with CNN and BiLSTM = six primary experiments. Text is lower-cased and tokenised with a regex that keeps punctuation, markdown symbols and **newlines (as a <nl> token)**; sequences are truncated/padded to 180 tokens. In the combined condition the prompt is capped at 60 tokens so a long prompt can never crowd the response out of the window. A seventh condition, *output_noformat*, strips markdown/list/code markers and collapses whitespace (RQ4 ablation).

Baselines and metrics. We report accuracy and **macro-F1** (robust to class imbalance), compared against uniform chance (1/K), the majority-class rule, and a TF-IDF (1–2-gram) + logistic-regression classifier that uses the same splits. A strong linear baseline tells us whether deep models add anything. Uncertainty: scores are mean ± std over 3 model seeds, and 95% CIs for macro-F1 come from a bootstrap that resamples whole question groups.

4. CNN and RNN Implementation

CNN (TextCNN). Embedding (96-d, padding index 0) → three parallel Conv1d layers with kernel widths 3, 5, 7 (96 filters each, 'same' padding) → ReLU → *masked* global max-pooling (padded positions cannot win the max) → concatenated 288-d vector → dropout 0.35 → linear layer. Kernels act as learned n-gram detectors for phrases such as stock openers and hedges.

RNN (BiLSTM). Embedding (96-d) → one-layer bidirectional LSTM (64 units per direction) run on *packed* sequences, so the final states reflect the last real token rather than trailing padding → concatenated forward/backward states (128-d) → dropout 0.35 → linear layer.

Training. Cross-entropy; AdamW (lr 0.002, weight decay 0.01); batch size 64; gradient-norm clipping at 1.0; at most 8 epochs with early stopping after 2 epochs without validation macro-F1 improvement; the data split is fixed (seed 42) and model initialisation / shuffling use the listed seeds; GPU when available.

5. Results: RQ1 and RQ2

Model	Input	Test accuracy	Test macro-F1	95% CI (macro-F1)
CNN	Input only	0.212 ± 0.003	0.184 ± 0.004	[0.164, 0.199]
CNN	Output only	0.555 ± 0.010	0.554 ± 0.011	[0.537, 0.582]
CNN	Input + output	0.547 ± 0.012	0.545 ± 0.012	[0.519, 0.564]
BiLSTM	Input only	0.206 ± 0.010	0.203 ± 0.011	[0.172, 0.210]
BiLSTM	Output only	0.473 ± 0.011	0.469 ± 0.007	[0.439, 0.484]
BiLSTM	Input + output	0.392 ± 0.014	0.397 ± 0.011	[0.388, 0.433]
Chance	–	0.200	–	
majority class (ChatGLM)	–	0.200	0.067	
TF-IDF + LogReg	Input only	0.225	0.225	
TF-IDF + LogReg	Output only	0.594	0.590	
TF-IDF + LogReg	Input + output	0.580	0.575	

Table 2. Held-out test results (results/metrics_summary.csv, baselines.csv). Mean ± std over seeds 42, 43, 44; the 95% CI is a question-group bootstrap on the seed-42 predictions only. Deep models vs. non-neural reference rows in italics.

RQ1: identifying the family from the response

With responses alone, the CNN reaches macro-F1 0.554 and the BiLSTM 0.469 (chance = 0.200 for 5 families); the CNN is higher by 0.085. The TF-IDF + logistic-regression reference scores 0.590, so the best neural model is not above this linear baseline (-0.036). Per-family precision, recall and F1 are in Table 3 and the confusion matrices in Figure 1.

Family	CNN P	CNN R	CNN F1	LSTM P	LSTM R	LSTM F1	Support
ChatGLM	0.46	0.35	0.40	0.35	0.31	0.33	320
Claude	0.88	0.73	0.80	0.66	0.74	0.70	320
GPT	0.46	0.52	0.49	0.39	0.53	0.45	320
Google	0.61	0.79	0.69	0.68	0.45	0.54	320
Llama-derived	0.43	0.43	0.43	0.30	0.29	0.30	320

Table 3. Per-family test metrics, output-only condition (P = precision, R = recall).

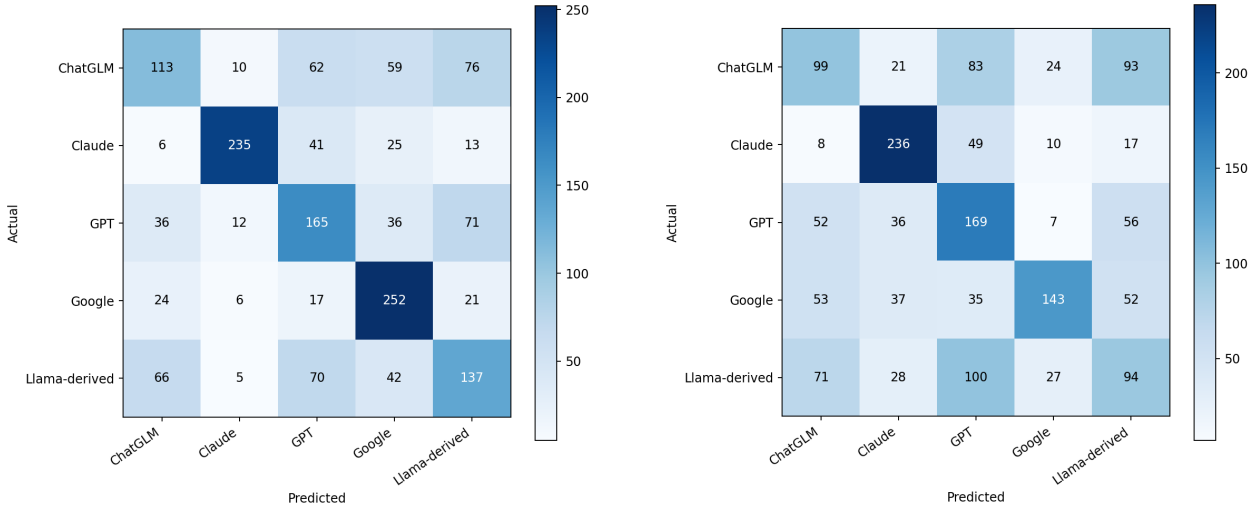


Figure 1. Confusion matrices (rows = actual family) for output-only CNN (left) and BiLSTM (right). Off-diagonal mass shows which families the models confuse.

RQ2: does the prompt help?

CNN: input-only 0.184 (-0.016 vs. chance), output-only 0.554, input+output 0.545 (-0.009 vs. output-only); BiLSTM: input-only 0.203 (+0.003 vs. chance), output-only 0.469, input+output 0.397 (-0.072 vs. output-only).

Input-only performance is at or near chance, which is what we expect by design: each Arena prompt is answered by two different models, so the prompt alone carries little information about which one answered. Any gain from the combined condition must therefore come from the response text, not from prompt-to-model correlations.

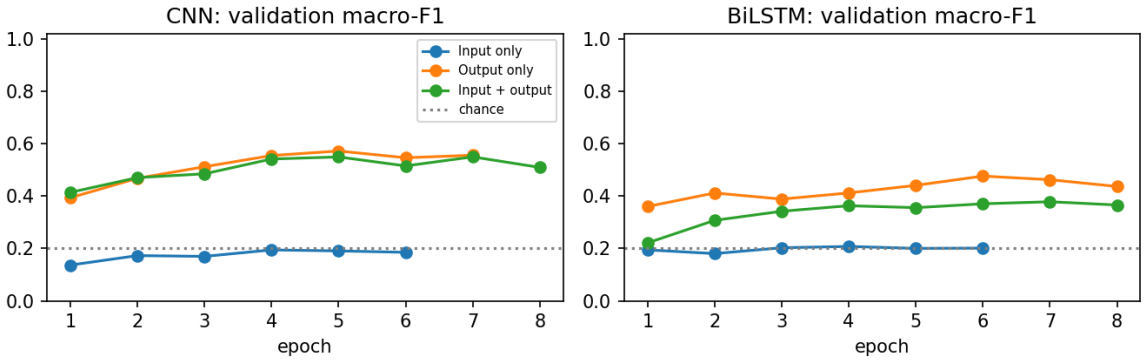


Figure 2. Validation macro-F1 per epoch (first seed). Early stopping restores the best epoch.

6. RQ4: What Distinguishes the Families?

We measure eight interpretable response properties: token count, lexical diversity (unique/total tokens), lexical diversity on the first 100 tokens (length-controlled, since the plain ratio falls as responses get longer), list-item lines, code-fence delimiters, newlines, mean token length and the count of assistant boilerplate phrases (“as an AI”, “I’m sorry”, …). We then (a) tabulate them by family, (b) train a gradient-boosting classifier on *only* these features and measure permutation importance, and (c) rerun the networks with all formatting removed.

Family	Tokens	TTR@100	List lines	Code fences	Newlines	AI phrases
ChatGLM	203.63	0.61	2.16	0.32	9.54	0.27
Claude	245.36	0.63	2.84	0.35	17.40	0.18
GPT	203.09	0.65	2.61	0.32	11.94	0.27
Google	268.71	0.58	3.33	0.47	14.75	0.13
Llama-derived	179.78	0.63	1.87	0.19	7.65	0.21

Table 4. Mean feature values per family (medians in results/rq4_features.csv).

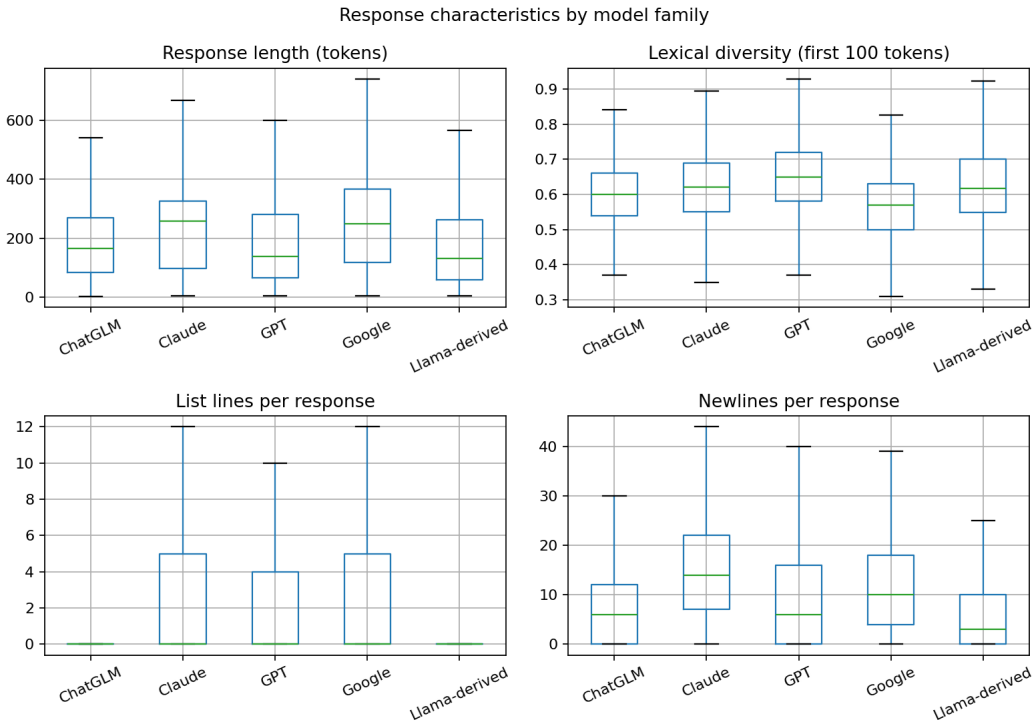


Figure 3. Distribution of four features by family (outliers hidden).

Largest observed differences: word count is highest for Google (268.7) and lowest for Llama-derived (179.8); bullet lines is highest for Google (3.3) and lowest for Llama-derived (1.9); newline count is highest for Claude (17.4) and lowest for Llama-derived (7.7). These are correlational: families were shown different prompts, so a family that looks "longer" may simply have received longer tasks.

A gradient-boosting model on these eight features alone reaches macro-F1 0.411 (accuracy 0.413). Permutation importance (drop in test macro-F1 when a feature is shuffled) ranks newline count (0.174), lexical diversity (0.089), word count (0.080) highest.

Formatting ablation (output macro-F1, original → formatting stripped): CNN: 0.554 → 0.446 (-0.108); BiLSTM: 0.469 → 0.426 (-0.043). A small drop means the networks rely mainly on wording; a large drop means list/markdown/layout conventions carry much of the signal.

RQ3: Generalisation across task types (extra credit)

Prompts are assigned to coding, math, writing or general by transparent keyword rules (DOMAINS in project.py); these are heuristic labels, not ground truth. For each domain we train output-only models on the other domains and test on the held-out one.

Model	Held-out domain	n	Out-of-domain macro-F1	In-domain macro-F1 (same rows, main model)
CNN	coding	927	0.499	0.592
BiLSTM	coding	927	0.412	0.462
CNN	general	5378	0.446	0.571
BiLSTM	general	5378	0.341	0.455
CNN	math	644	0.407	0.484
BiLSTM	math	644	0.412	0.476
CNN	writing	1051	0.527	0.533
BiLSTM	writing	1051	0.437	0.487

Table 5. Leave-one-domain-out results (results/rq3.csv). The in-domain column is computed on the main model's test predictions for that domain; the two models differ in training data size, so compare gaps cautiously.

Reading the table. Averaged over domains, held-out performance is below in-domain performance by 0.076 macro-F1 for the CNN and 0.069 macro-F1 for the BiLSTM. The largest drop is CNN on *general* (0.571 to 0.446); the smallest is CNN on *writing* (0.533 to 0.527). Every held-out score stays well above chance (0.200), so part of the signal is not tied to one task type. Two cautions apply. First, the *general* domain contains most prompts, so holding it out removes most of the training data; its large drop mixes domain shift with a smaller training set. Second, the in-domain column comes from the main model trained on all domains, so it also benefits from seeing more data. A drop therefore does not prove the model learned task-specific patterns, and the small drops (e.g. writing) do not prove a task-independent fingerprint. The RQ3 scores are from a single seed (42), unlike the main table.

7. Limitations

(1) Family labels merge many checkpoints and fine-tunes. (2) The data are from 2023 and reflect Arena users' tasks; results do not transfer automatically to current models. (3) Truncation at 180 tokens hides length and late-response style; padding is masked, so the CNN cannot see raw length at all. (4) Multi-turn conversations are flattened into one prompt and one response string. (5) Scores come from one fixed split; seed variation only covers model initialisation. (6) Moderation filtering and the English-only filter further narrow the population. (7) A classifier that works on this dataset may have learned collection artefacts rather than model identity.

8. Lessons and Experience

Methodology. Grouping by *question_id* is essential when several answers share a prompt; baselines (chance, majority, TF-IDF) are needed before a neural score means anything; macro-F1 exposes weak small classes that accuracy hides; and an attractive feature table is not a causal explanation — the ablation is what tests it.

Implementation. Several details silently change results: running an LSTM over padded batches without packing lets trailing padding overwrite the final state; max-pooling over padded positions lets padding compete with real tokens; naive truncation of "prompt + response" can delete the response entirely; and one ambiguous model name (GPT4All) would have been mislabelled as an OpenAI model by a substring rule. Each is handled in *project.py*.

The biggest surprise was that the simplest model was the strongest. A TF-IDF plus logistic-regression baseline scored macro-F1 0.590 on responses alone, above my best neural model (CNN, mean 0.554 over three seeds) and well above the BiLSTM (0.469). I had expected the networks to win, so I treated this as a result to explain rather than hide. The likely reasons are that the training set is small (5,600 rows), that truncation at 180 tokens throws away information the linear model still sees, and that much of the signal is in specific words and phrases that n-gram counts capture directly. Two other results were equally clear and stable across seeds. The prompt alone is useless for identifying the family (macro-F1 0.18 to 0.20, at chance), which makes sense because every Arena prompt is answered by two different models, and adding the prompt to the response did not help (CNN 0.554 to 0.545, BiLSTM 0.469 to 0.397). The families were not equally easy to spot either: in the seed-42 models Claude was the most

identifiable (CNN F1 0.80), while ChatGLM (0.40) and the merged Llama-derived group (0.43) were hardest, which fits the fact that Llama-derived mixes several fine-tunes.

Explaining why the classifiers work was less clean than showing that they do. Newline count was the most important feature in the gradient-boosting analysis (permutation importance 0.174), and removing markdown, lists and line breaks lowered the neural scores (CNN 0.554 to 0.446, BiLSTM 0.469 to 0.426), but the size of that drop was unstable. The CNN's formatting-stripped score ranged from 0.384 to 0.490 across the three seeds, so the average drop of about 11 points is far less certain than a single run suggested; had I stopped at one seed I would have reported a drop of 7 points. The TF-IDF model barely changed (0.590 to 0.566), and all models stayed well above chance, so wording carries most of the signal and layout adds some of it. I also cannot say how much of either is true model style rather than the kinds of prompts each model happened to receive. The leave-one-domain-out experiment points the same way: performance dropped by about 0.07 macro-F1 on unseen task types but stayed well above chance. Those drops are confounded by training-set size (holding out the large "general" group removes most of the data), the domains come from keyword rules, and RQ3 used one seed, so I treat it as suggestive only.

On the process side, long runs were interrupted more than once, which taught me to make experiments recoverable. I ran each seed as its own job, rebuilt the seed-42 test predictions from saved checkpoints and checked that they reproduced the reported accuracies exactly (0.56375 and 0.463125) before running RQ3 on top of them, and re-ran only the conditions that were missing. The code details that mattered most were the ones that silently change results: masking padding in max-pooling, packing sequences for the LSTM, capping the prompt in the combined condition, and grouping both answers to one question into the same split. With more time I would use more than three seeds (the ablation shows three is still noisy), balance the families more carefully, split the Llama-derived group by checkpoint, test on a more recent dataset that includes current model families, and use real task labels instead of keyword rules for RQ3.

9. Reproduction and References

Repository: *[GitHub URL]*. Contents: project.py, make_report.py, requirements.txt, README.md, this report. Run python project.py --data data/chatbot_arena.parquet --out results --seeds 42 43 44 --rq3 then python make_report.py --team "...". Raw data and model weights are excluded (see .gitignore). Use of LLM assistants: Claude was used for brainstorming, code review, running and debugging the experiments, and drafting report text; the author is responsible for checking and explaining all code, results and conclusions.

Zheng, L. et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv:2306.05685.
LMSYS (2023). Chatbot Arena Conversation Dataset Release. lmsys.org/blog/2023-07-20-dataset/